

前后端联合架构与联调手册

适用代码：E:\new\my_harmony 鸿蒙业务前端 + E:\goTest Go 后端宿主

核对日期：2026-07-21

文档定位：架构解释、接口参考、联调操作和故障排查

0. 阅读前先确认

当前系统不是一个 HAP 内的“前端调用本地 Go 函数”，而是两个独立 HarmonyOS 应用：

应用	工程目录	Bundle Name	主要职责
鸿蒙业务前端	E:\new\my_harmony	com.nutpi.My_Project	页面、交互、状态管理、HTTP/WebSocket 客户端
Go 后端宿主	E:\goTest	com.nutpi.gotest	启动 Go 动态库、HTTP/WebSocket、CTCP、SQLite、OPC UA、Modbus

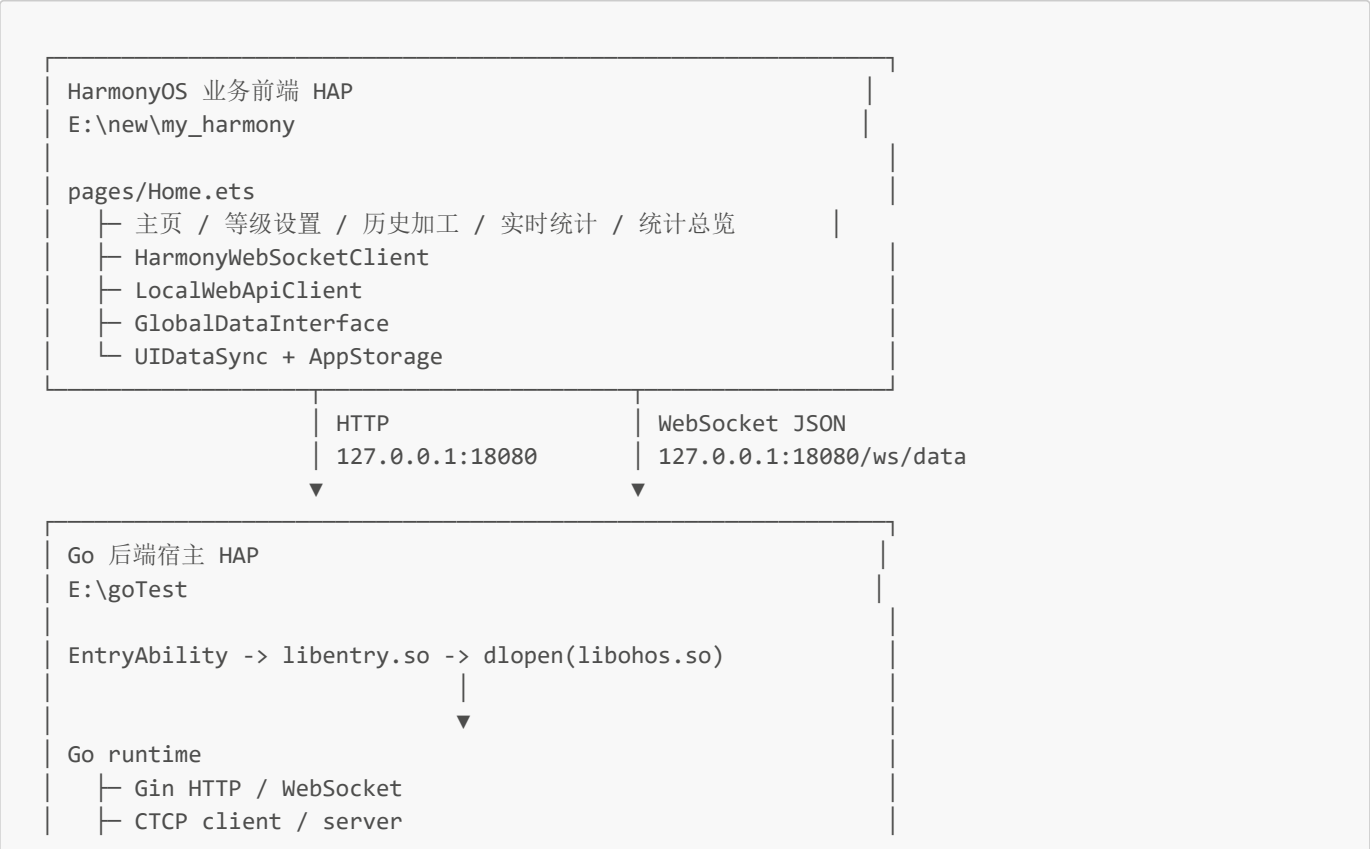
两个应用通过设备本机的 localhost 通信：

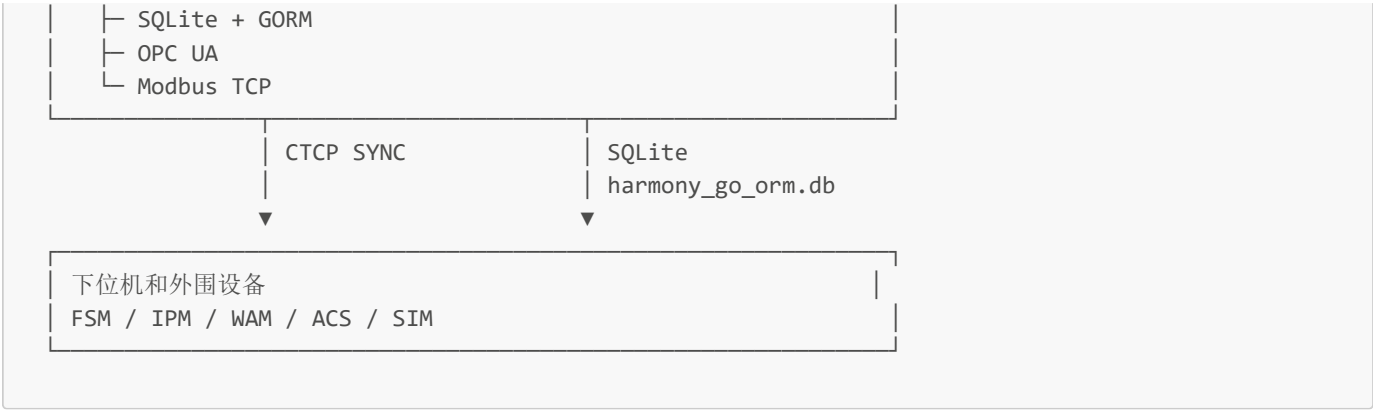
- HTTP：http://127.0.0.1:18080/
- WebSocket：ws://127.0.0.1:18080/ws/data

这意味着：

1. 业务前端不会自动拉起 Go 后端应用。
2. Go 后端未启动或 18080 监听失败时，业务前端无法自行恢复服务。
3. E:\goTest\entry 是后端宿主和调试页面，不是实际业务前端。
4. E:\new\my_harmony 当前没有业务 Native Bridge，主要通过 HTTP/WebSocket 使用 Go 服务。
5. 两个应用的构建、签名、安装和生命周期相互独立。

1. 总体架构





1.1 职责边界

业务前端负责：

- ArkUI 页面布局和用户操作。
- 表单草稿、弹窗、导航和页面生命周期。
- 将 Go 返回的 JSON 映射为 ArkTS 结构。
- 在 `GlobalDataInterface` 中维护前端运行态。
- 通过 `UIDataSync` 将汇总数据写入 `AppStorage`。
- 发起查询、配置保存、测试命令和设备控制。

Go 后端负责：

- 在 HarmonyOS 中长期运行 Go runtime。
- 将 HTTP/WebSocket 请求转换为设备 CTCP 指令。
- 监听下位机反向连接并解析二进制结构体。
- 缓存 `StGlobal`、`StStatistics`、重量、波形和出口状态。
- 将设备数据转换为 JSON 并推送给业务前端。
- 每 3 秒聚合实时统计并写入 SQLite。
- 提供 OPC UA 和 Modbus TCP 服务。

C++ NAPI 层只负责：

- 加载 `libohos.so`。
- 解析 Go 导出符号。
- 将少量生命周期函数暴露给后端宿主 ArkTS。
- 转换字符串和基础数值，不承载业务数据模型。

2. 工程目录地图

2.1 鸿蒙业务前端

```
E:\new\my_harmony
├─ AppScope/
│   └─ app.json5          应用 Bundle、版本、图标
├─ entry/
│   └─ src/main/ets/
│       ├── entryability/EntryAbility.ets  前端生命周期入口
│       ├── pages/Home.ets                 主页面容器和全局导航
│       ├── pages/home/                   主页业务组件和管理器
│       ├── pages/level/                  等级设置
│       ├── pages/quality/                品质设置
│       ├── pages/history/                历史查询
│       ├── pages/realtime/               实时统计
│       └─ pages/endprocessing/           结束加工
```

├── pages/overview/	统计总览
├── components/	通用表格、图表、弹窗、布局
├── protocol/	数据结构、状态、HTTP 客户端、发送门面
├── utils/network/	WebSocket 客户端
└── src/main/module.json5	Ability 和权限
├── tools/	静态验证脚本
└── README.md	历史架构说明，部分内容已过时

前端主要入口：

主题	文件	关键符号
Ability 生命周期	entry/src/main/ets/entryability/EntryAbility.ets	onCreate()、onDestroy()
主页面和全局导航	entry/src/main/ets/pages/Home.ets	build()、页面 Builder
导航配置	entry/src/main/ets/pages/home/HomePageNavigation.ets	createHomeNavigationButtons()
WebSocket	entry/src/main/ets/utils/network/HarmonyWebSocketClient.ets	start()、connect()、handleTextMessage()
HTTP	entry/src/main/ets/protocol/LocalWebApiClient.ets	post()、request()
命令门面	entry/src/main/ets/protocol/ConfigSender.ets	等级、系统、WAM、IPM 命令入口
全局运行态	entry/src/main/ets/protocol/GlobalDataInterface.ets	配置和统计快照
页面状态同步	entry/src/main/ets/protocol/UIDataSync.ets	start()、stop()

2.2 Go 后端宿主

E:\goTest	
├── AppScope/	
│ └── app.json5	后端宿主 Bundle 和版本
├── entry/	
│ ├── src/main/ets/	
│ │ ├── entryability/EntryAbility.ets	启动和停止 Native 服务
│ │ └── pages/Index.ets	后端调试页面
│ ├── src/main/cpp/	
│ │ ├── napi_init.cpp	NAPI 和 dlopen/dlsym
│ │ └── include/libohos.h	Go c-shared 生成的头文件
│ └── libs/arm64-v8a/	
│ ├── libohos.so	Go 动态库
│ └── liblog.so	日志兼容库
├── go/ohos/	
│ ├── main.go	Go C ABI 导出
│ ├── build_ohos.ps1	HarmonyOS 交叉编译脚本
│ ├── Tcp/	
│ │ ├── api.go	稳定对外入口
│ │ ├── client/	CTCP/TCP 下发
│ │ ├── protocol/	线协议结构体
│ │ ├── events/	日志和发布接口
│ │ ├── state/	第一批运行时快照
│ │ └── internal/runtime/	HTTP、WS、接收、缓存、统计、落库
│ ├── database/	GORM、迁移、模型和 HTTP API
│ └── opcua/	OPC UA Server

```
| └─ modbus/  
| └─ docs/  
| └─ tools/
```

Modbus TCP Server
技术说明和测试报告
稳定性、性能和静态验证工具

3. 应用启动和停止

3.1 Go 后端宿主启动顺序

后端宿主 `EntryAbility` 加载 `pages/Index` 后，通过 `setTimeout(..., 0)` 调用 `startNativeServices()`。

实际顺序如下：

```
EntryAbility.onWindowStageCreate  
-> loadContent('pages/Index')  
-> startNativeServices()  
    1. initOrm()  
    2. startWebSocketServer()  
    3. startOpcuaServer()  
    4. startModbusServer()  
    5. startTcpServer()  
    6. startTcpServerLogPolling()
```

各步骤作用：

1. `initOrm()`

- 数据库路径： `${context.databaseDir}/harmony_go_orm.db`。
- ArkTS 调用 `libentry.so` 的 `initOrm(path)`。
- C++ 调用 Go 的 `GoInitORMWithPath`。
- Go 使用 GORM + SQLite 打开文件库、迁移表并应用 SQL migration。

2. `startWebSocketServer()`

- 调用 Go `GoStartServer`。
- 监听 `127.0.0.1:18080`。
- 注册 `/ping`、数据库 API、WebSocket 和 About API。

3. `startOpcuaServer()`

- 启动 OPC UA Server，默认端口 `4840`。

4. `startModbusServer()`

- 启动 Modbus TCP Server，默认端口 `5020`。

5. `startTcpServer()`

- 启动 CTCP `1127` 和 `1128` 两个监听端口。
- 加载本地出口、等级辅助设置和出口屏配置。
- 启动出口屏广播、统计推送和实时落库节奏。

6. `startTcpServerLogPolling()`

- 每 250 ms 从 Go 日志环形缓冲取数据。
- 将 Go 运行日志转发到 Harmony HiLog。

3.2 C++ NAPI 如何加载 Go

```
ArkTS: testNapi.startServer()
-> libentry.so: NapiStartServer()
-> LoadGoApi()
    dlopen("libohos.so", RTLD_NOW | RTLD_GLOBAL)
    fallback: /data/storage/el1/bundle/libs/arm64/libohos.so
    dlsym("GoStartServer")
-> GoStartServer()
-> tcp.StartServer()
-> runtime.StartServer()
```

`LoadGoApi()` 会一次性检查主要导出符号。任一必需符号缺失，NAPI 会保留 `lastError`，ArkTS 可通过 `nativeLastError()` 读取。

3.3 业务前端启动顺序

```
EntryAbility.onCreate
-> HarmonyWebSocketClient.start()
-> UIDataSync.start()

EntryAbility.onWindowStageCreate
-> loadContent('pages/Home')
-> 设置全屏窗口
```

WebSocket 连接成功后，前端立即：

1. 对 4 个 FSM 分别发送 `requestStGlobal`。
2. 发送 `requestHomeStats`。
3. 发送 `requestStatistics`。

`requestStGlobal` 会让 Go 向目标 FSM 的 1279 端口发送 `DISPLAY_ON(0x0019)`。FSM 随后通过新的反向连接把配置和统计发到 HC 的 1128 端口。

3.4 推荐启动顺序

1. 安装并启动 Go 后端宿主
2. 确认 `http://127.0.0.1:18080/ping` 可用
3. 确认 1127/1128 监听成功
4. 启动业务前端
5. 确认 WebSocket 已连接
6. 确认前端已发 `requestStGlobal`
7. 确认 FSM 回推 `StGlobal` 和 `Statistics`

业务前端先启动也不会崩溃，但会每 3 秒持续重连 WebSocket。后端起来后，下一次重连才建立完整链路。

3.5 停止顺序

业务前端 `onDestroy()`：

1. 发送 `{ "type": "close_client" }`。
2. 关闭 WebSocket。
3. 停止 `UIDataSync`。

WebSocket 连接断开时，Go 会检查本连接记录的持续性 On 命令，例如波形、数据追踪、连续采集和测试模式，并自动发送对应 Off 命令。

后端宿主 `onDestroy()`：

- 1. 停止 CTCP Server。
- 2. 停止 Modbus。
- 3. 停止 OPC UA。
- 4. 停止 HTTP/WebSocket Server。

4. 端口和设备地址参考

端口	监听方	方向	用途
18080	Go 后端	前端双向	localhost HTTP + WebSocket
1127	Go 后端	IPM/ACS -> HC	图像、出口状态类数据
1128	Go 后端	FSM/WAM -> HC	StGlobal、StStatistics、等级、重量、波形
1279	FSM	HC -> FSM	DISPLAY_ON、等级、系统配置、控制命令
1289	IPM	HC -> IPM	采样、快门、白平衡、关机、图像命令
1299	WAM	HC -> WAM	重量配置、清零、测试、波形、追踪
4127	ACS	HC -> ACS	ACS 命令
1129	SIM	HC -> SIM	模拟设备命令
4840	Go 后端	外部客户端 -> HC	OPC UA Server
5020	Go 后端	外部客户端 -> HC	Modbus TCP Server

Go 的 CTCP 目标地址解析规则：

- 默认网段：192.168.0.x。
- FSM1 `destId=0x0100` -> 192.168.0.16:1279。
- FSM2 `destId=0x0200` -> 192.168.0.32:1279。
- IPM 命令号位于 0x2000~0x2FFF，目标端口改为 1289。
- WAM 命令号位于 0x0110~0x1FFF，目标端口改为 1299。
- ACS 命令号 `>=0x9000`，目标为 192.168.0.13:4127。
- SIM 命令号位于 0x7000~0x7102，目标为 192.168.0.14:1129。

5. HTTP 接口参考

5.1 客户端配置

前端默认地址：

```
http://127.0.0.1:18080/
```

可通过 `AppStorage.WEB_SERVICE_URL` 覆盖。连接和读取超时均为 15 秒。

业务请求统一使用：

- 方法：POST
- `Content-Type: application/json`
- 请求体：JSON

业务 `/Api/*` 接口通常返回：

```
{
  "returnCode": 1,
  "returnMessage": "",
  "data": "... "
}
```

前端规则：

- HTTP 状态码不是 2xx：直接失败。
- returnCode != 1：按业务失败处理。
- data 通常是 JSON 字符串，前端再做第二次解析。

5.2 路由组

路由组	主要用途	后端文件
/ping	Go HTTP 存活检查	Tcp/internal/runtime/server.go
/orm、/orm/init	ORM 状态和初始化	database/orm.go
/Api/FruitInfo/*	批次、水果、加工状态、运行时间	database/fruit_info_api.go 等
/Api/FruitProcessInfo/*	加工曲线和时间范围查询	database/fruit_process_info_api.go
/Api/SysConfig/*	系统配置查询和保存	database/sys_config_api.go
/Api/Fault/*	故障基础信息和故障记录	database/fault_api.go
/Api/SysFruitInfo/*	子系统加工数据	database/sys_fruit_info_api.go
/Api/DeviceConfig/*	云端设备配置列表、下载、删除	database/device_config_cloud_api.go
/Api/PrintTemplate/*	打印模板、预览、Excel 导出	database/print_template_api.go
/Api/Version/*	版本检查和下载	database/version_update_api.go
/Api/DeviceInfo/*	设备信息、地点、注册	database/device_info_api.go
/Api/About/*	关于页面和版本信息	Tcp/internal/runtime/about_info_api.go
/Api/Debug/GoLogs	Go 后端日志环形缓冲	Tcp/internal/runtime/websocket.go

注意：opcua 和 modbus 包中虽然保留了 HTTP 控制路由函数，但当前 runtime.newRouter() 没有注册这些路由。当前宿主是在启动时直接启动 OPC UA 和 Modbus 服务。

6. WebSocket 接口参考

6.1 连接参数

- 默认地址：ws://127.0.0.1:18080/ws/data
- 前端覆盖键：AppStorage.WEBSOCKET_CLIENT_URL
- 前端断线重连：3 秒
- Go Ping 周期：30 秒
- Go Pong 等待：70 秒
- 单连接发送队列：32 帧
- Hub 广播队列：64 帧
- 最大消息：约 4 MiB

Go 同时注册 /ws 和 /ws/data，业务前端当前使用 /ws/data。

6.2 后端推送帧

```
{
  "type": "data",
  "topic": "statistics",
  "data": {},
  "requestId": "",
  "at": 1780000000000
}
```

命令结果使用：

```
{
  "type": "commandAck",
  "topic": "saveLevelData",
  "data": {
    "result": 0,
    "ok": true,
    "command": "saveLevelData",
    "cmdId": 81,
    "destId": 256,
    "payloadBytes": 0,
    "message": "sent",
    "requestId": "saveLevelData-1"
  },
  "at": 1780000000000
}
```

6.3 实时 Topic

Topic	数据	前端处理
stglobal	FSM 完整配置快照	映射为 StGlobal，打开配置门控
statistics	StStatistics 快照	更新子系统统计和实时服务
homeStats	Go 聚合后的主页指标	更新 KPI、效率、曲线
grade	单果分级实时数据	更新实时水果信息
exitInfos	出口名称、模式、箱型	更新出口配置和卡片
exitDisplay	出口屏显示配置	更新出口屏状态
exitAdditionalText	出口屏附加文本	写入前端存储
levelAuxConfig	等级精度、装箱量、重量标准等	更新等级辅助配置
fruitTypeConfig	水果类型映射	更新水果类型配置
projectSettingsSchemeFile	项目设置方案文件	触发方案导入/导出流程
weightGlobal	WAM 全局重量配置	更新重量配置快照
weightInfo	重量结果	更新实时重量状态
waveInfo	波形数据	更新波形捕捉服务
ipmImage	IPM 图像帧	更新图像流服务

6.4 前端控制消息

控制消息以 `type` 分发。主要分组：

分组	常见 Type
初始化查询	<code>requestStGlobal</code> 、 <code>requestHomeStats</code> 、 <code>requestStatistics</code>
等级/品质	<code>saveLevelData</code> 、 <code>saveQualityData</code> 、 <code>dropdata</code> 、 <code>clearExitGrades</code>
FSM 配置	<code>saveSysConfig</code> 、 <code>saveParasInfo</code> 、 <code>saveGlobalExitInfo</code> 、 <code>saveExitInfo</code>
加工控制	<code>clearData</code> 、 <code>endProcess</code> 、 <code>saveParasToFlash</code> 、 <code>exitClear</code>
FSM 测试	<code>fsmTestCupOn/Off</code> 、 <code>fruitGradeInfoOn/Off</code> 、 <code>testValve</code>
WAM	<code>wamGetInfo</code> 、 <code>wamWeightReset</code> 、 <code>wamWaveFormOn/Off</code> 、 <code>wamDataTrackingOn/Off</code>
IPM	<code>ipmSingleSample</code> 、 <code>ipmContinuousSampleOn/Off</code> 、 <code>ipmShutterAdjustOn/Off</code>
出口屏	<code>saveExitDisplay</code> 、 <code>setExitDisplayEnabled</code> 、 <code>saveExitScreenSettings</code>
本地配置	<code>saveLevelAuxConfig</code> 、 <code>saveFruitTypeConfig</code> 、 <code>saveProjectSettingsScheme</code>
数据查询	<code>queryFruitInfo</code> 、 <code>querySortLog</code> 、 <code>querySysFruitInfo</code>
项目认证	<code>createProjectPassword</code> 、 <code>validateProjectPassword</code>

6.5 CommandAck 的正确含义

`commandAck.ok=true` 表示 Go 已完成当前命令处理，并且底层发送调用返回成功。它不等价于 FSM 已应用配置。

必须区分四层状态：

层级	证明了什么	没有证明什么
WebSocket <code>send()</code> 成功	消息进入当前 WS 连接	Go 已解析、设备已收到
<code>commandAck.ok=true</code>	Go 处理成功，TCP 发送调用成功	设备已应用配置
收到新的 <code>StGlobal</code>	FSM 回传了配置快照	当前页面已消费正确快照
页面最终更新	前端状态链完成	后续不会被旧快照覆盖

设备回读是配置是否真正生效的权威证据。

7. CTCP 协议和设备通信

7.1 SYNC 帧

当前设备通信使用固定 SYNC 头：

```
"SYNC" 4 字节
+ srcId 4 字节
+ dstId 4 字节
+ cmdId 4 字节
+ payload N 字节
```

Go 接收顺序：

```
recvCTCPSync()
-> recvCTCPCommand()
-> 校验 dstId == HC_ID(0x1000)
```

```
-> recvCTCPPayload()  
-> handleCommandPayload()
```

7.2 常用设备 ID

对象	ID 示例
HC 上位机	0x1000
FSM1	0x0100
FSM2	0x0200
FSM3	0x0300
FSM4	0x0400
WAM1	0x01D0

7.3 常用命令

命令	ID	用途
DISPLAY_OFF	0x0000	关闭显示/连接
CLEAR_DATA	0x0001	数据清零，不结束批次
TEST_CUP_ON/OFF	0x0006/0x0007	果杯测试
FRUIT_GRADE_ON/OFF	0x000F/0x0010	实时分级开关
MOTOR_ENABLE	0x0016	电机使能
SAVE_PARAS	0x0018	保存参数到 Flash
DISPLAY_ON	0x0019	请求 FSM 开始回推配置和统计
EXIT_CLEAR	0x001F	出口清框
SYS_CONFIG	0x0050	系统/硬件配置
GRADE_INFO	0x0051	等级配置
EXIT_INFO	0x0052	出口配置
PARAS_INFO	0x0054	通道参数
GLOBAL_EXIT_INFO	0x0058	全局出口参数
MOTOR_INFO	0x005C	电机参数
COLOR_GRADE_INFO	0x005D	品质配置
DENSITY_INFO	0x005E	密度配置
FSM_CONFIG	0x1000	FSM 回推 StGlobal
FSM_STATISTICS	0x1001	FSM 回推 StStatistics

7.4 1127/1128 接收和分发

StartCTCPServer() 同时监听：

- 0.0.0.0:1127，命名为 image。
- 0.0.0.0:1128，命名为 stat。

关键分支：

回包	Go 处理
FSM_CONFIG	解析 StGlobal，更新多组缓存，转换 JSON，推送 stglobal
FSM_STATISTICS	解析统计，标准化子系统 ID，缓存并触发统计/落库节奏
FSM_GRADE_INFO	解析单果分级，推送 grade
FSM/WAM 重量	解析 StWeightResult，推送 weightInfo
FSM/WAM 波形	解析 StWaveInfo，推送 waveInfo
WAM 全局重量	解析 StWeightGlobal，推送 weightGlobal
IPM 图像	封装图像帧，推送 ipmImage
其他命令	保存原始 payload 并写诊断日志

8. 前端状态模型

8.1 数据进入页面的主链路

WebSocket 文本帧

-> HarmonyWebSocketClient.handleTextMessage()

-> 按 topic 选择 JSON Mapper

-> ArkTS StGlobal / StStatistics / StWeightInfo

-> GlobalDataInterface

-> 监听器

-> UIDataSync / RealtimeStatsService / 页面 Manager

-> AppStorage / @State / @StorageLink

-> ArkUI 组件重绘

8.2 GlobalDataInterface

它是前端运行态中枢，主要维护：

- 每个子系统的 StGlobal。
- 每个子系统的 StStatistics。
- 配置是否就绪。
- 设备等级回读是否就绪。
- 重量、设备状态、单果分级和业务汇总。
- 合并后的全局统计与分子系统统计。

重要门控：

- isConfigReadyForSubsys()：当前 FSM 配置是否完整。
- isDeviceGradeEchoReady()：当前等级是否已经得到设备回读确认。
- 统计先到而配置未到时，前端暂存统计并限频补发 requestStGlobal。

8.3 UIDataSync

UIDataSync 将全局运行态整理为页面易消费的键，包括：

- 实时产量。
- 本批重量和个数。
- 平均重量。
- 分选速度和效率。

- 开始时间和运行时间。
- 出口计数、出口重量、出口百分比。
- 加工曲线点。
- 当前加工状态。

出口类数据有刷新节流，状态过期后会按 stale 规则回落，避免页面长期显示旧值。

8.4 四级状态源

页面编辑草稿

- > 前端本地状态
- > Go 后端缓存
- > FSM 实际配置和设备回读

越靠下越接近权威状态。排查“页面变了但设备没变”时，必须先确认页面当前展示的是哪一级。

9. 关键业务链路

9.1 首次连接和配置同步

业务前端 WS open

- > 4 × requestStGlobal(fsmId)
- > Go RequestStGlobalFromFSM()
- > CTCP DISPLAY_ON 0x0019 -> FSM:1279
- > FSM 新建反向连接 -> HC:1128
- > FSM_CONFIG / StGlobal
- > Go 解析、缓存、推送 topic=stglobal
- > 前端映射并 updateGlobalConfigFromDeviceEcho()
- > configReady=true
- > 页面开始消费完整配置和统计

重点：FSM 回包不是原 1279 连接上的同步返回，而是 FSM 向 HC 1128 发起的新连接。

9.2 实时统计链路

FSM

- > HC:1128 / FSM_STATISTICS(0x1001)
- > parseStStatisticsPayload()
- > cacheStStatisticsForSpeed()
- > 每秒 publishLatestStStatisticsSpeed()
 - ├ topic=statistics
 - └ topic=homeStats
- > HarmonyWebSocketClient
- > GlobalDataInterface / RealtimeStatsService
- > UIDataSync / 页面表格和图表

统计流到达时还会触发 maybeSaveRealtimeStatistics()。数据库写入有独立的 3 秒节流。

9.3 等级配置保存链路

LevelContent / 等级设置页面

- > ConfigSender
- > HarmonyWebSocketClient.sendGradeInfoCommand('saveLevelData')

```

-> JSON: fsmId + grade + levelAuxConfig + gradeNameTexts
-> Go handleIncoming()
-> handleGradeInfoData()
-> SendGradeInfoData()
    1. 保存 levelAuxConfig
    2. 校验 grade 非空
    3. 必要时合并已缓存出口位
    4. GBK 固定槽名称编码
    5. 校验 StGradeInfo wire size
    6. 发送 0x0051 -> FSM:1279
    7. 返回 commandAck
    8. 0.3s、1.5s、3s 三次请求 StGlobal
-> FSM 回推新的 StGlobal
-> 前端设备回读就绪

```

等级保存默认不阻塞页面等待设备 ACK。前端会登记 ACK 用于日志和诊断，但最终仍以设备回读为准。

9.4 品质配置保存链路

品质配置使用相同的 `StGradeInfo` 数据结构，但命令为 `0x005D`，WebSocket Type 为 `saveQualityData`。

与等级保存不同，品质保存会等待设备命令结果。失败时不应把本地草稿当成已确认配置。

9.5 硬件设置与等级能力

硬件能力来自 `StGlobal.Sys`，等级规则来自 `StGlobal.Grade`：

硬件设置保存

```

-> saveSysConfig
-> Go 发送 SYS_CONFIG(0x0050)
-> Go 延时请求 StGlobal
-> FSM 回推 StGlobal.Sys

```

等级页面

```

-> 读取当前子系统 StGlobal.Sys 的能力开关
-> 决定可见/可编辑的分类方式
-> 编辑结果写入 StGlobal.Grade 对应字段

```

这两部分不是同一个数据源。硬件能力和等级规则之间的页面联动属于前端状态和页面门控职责；Go 负责分别下发并回推权威配置。修改联动时不能把 `Sys.*ClassifyType` 当作等级表的 `Grade.NClassifyType`。

9.6 HTTP 查询链路

页面/Service

```

-> LocalWebApiClient
-> POST /Api/...
-> Gin handler
-> getInitializedFileORMDB()
-> GORM 查询
-> ApiEnvelope
-> 前端解析 envelope.data
-> 页面 Manager / @State

```

适用于水果信息、历史加工、故障、打印模板、版本、设备信息等请求响应业务。

9.7 数据清零和结束加工

两者语义不同：

操作	批次	数据库	统计基线
数据清零 clearData	保留当前批次和开始时间	当前批次累计值清零，等级/出口/子系统明细删除	清除旧统计缓存并重新累计
结束加工 endProcess	结束当前批次	标记批次完成，后续增量形成新批次	暂停保存，等待统计归零或新增量

Go 为数据清零保留 90 秒识别窗口。窗口内观察到设备累计计数下降时，按“批内清零”处理，不错误拆成新批次。

10. SQLite 和实时落库

10.1 初始化

数据库文件：

<后端应用 databaseDir>/harmony_go_orm.db

初始化过程：

```
InitORMWithPath
-> resolveORMDSN
-> gorm.Open(sqlite)
-> 文件库启用 WAL + synchronous=NORMAL
-> migrateORMTables
-> applySQLMigrations
-> seedORMDevices
```

WAL 用于降低每 3 秒实时事务与 HTTP 查询互相阻塞的概率。

10.2 实时保存条件

maybeSaveRealtimeStatistics(now) 只有在以下条件满足时才写库：

- 1. 已收到有效 StGlobal。
- 2. 当前没有另一个保存事务执行中。
- 3. 距离上次保存至少 3 秒。
- 4. 当前统计可以构造出有效保存输入。
- 5. 结束加工后的暂停状态已经满足恢复条件。

10.3 单次事务内容

database.SaveRealtimeFruitInfo() 在同一个事务中：

- 1. 查找当前未完成批次。
- 2. 判断设备计数器是否递减。
- 3. 更新已有批次或创建新批次。
- 4. 替换等级明细 TbGradeInfo。
- 5. 替换出口明细 TbExportInfo。
- 6. 替换子系统明细 TbSysFruitInfo。
- 7. 写加工过程分钟/时间表。

单次保存超过 500 ms 会记录慢保存日志；正常情况下每 20 次输出一本次、平均和最大耗时。

11. 业务前端页面架构

11.1 主导航

当前导航项：

- 主页
- 等级设置
- 水果信息
- 载入程序
- 历史加工
- 实时统计
- 统计总览
- 结束加工
- 打印
- 更多

Home.ets 使用 Builder 按当前页面延迟创建组件，避免所有复杂页面在应用启动时同时实例化。

11.2 等级表实现

HarmonyOS ArkUI 没有完整的官方 Table 组件。项目中的表格是自定义组件：

- CompositeGradeTable：等级矩阵。
- ScrollableDataTable：等级统计表。
- GridTable：通用规则表格。
- LevelDataTable、QualityDataTable：业务页面表格。

内部通常使用：

```
Scroll
-> Column
  -> Row + Text 表头
  -> List + LazyForEach 数据行
    -> ForEach + Text 单元格
```

表格效果来自固定宽高、边框、背景色、滚动和虚拟列表，不是 Canvas 或第三方 Table 控件。

12. Go 后端模块边界

12.1 当前已拆出的包

包	职责
Tcp/client	普通 TCP、CTCP 下发、目标地址解析
Tcp/protocol	线协议结构体和布局检查
Tcp/events	日志缓冲和 WebSocket 发布接口
Tcp/state	StGlobal、等级、出口、Motor 等快照
Tcp/internal/runtime	当前仍强耦合的服务编排和业务运行时
database	GORM、表模型、迁移、HTTP API、实时保存
opcua	OPC UA Server
modbus	Modbus TCP Server

12.2 依赖原则

- 线协议结构体放 `protocol`，不要依赖业务包。
- 设备下发放 `client`，不要反向依赖 `runtime`。
- 运行时快照逐步收敛到 `state`。
- 数据库写入通过 DTO 边界进入 `database`。
- WebSocket 不应长期承担所有业务状态。
- `internal/runtime` 是过渡层，不代表最终目录设计。

13. 构建与部署

13.1 后端 Go 动态库

前置条件：

- Go 工具链。
- HarmonyOS Native SDK。
- `OHOS_SDK` 指向 SDK 的 OpenHarmony 根目录。
- 当前脚本只支持 `arm64-v8a`。

执行：

```
Set-Location E:\goTest
$env:OHOS_SDK = 'D:\harmSdk\command-line-tools\sdk\default\openharmony'
.\go\ohos\build_ohos.ps1
```

脚本会：

1. 必要时构建 `liblog.so`。
2. 设置 `GOOS=android`、`GOARCH=arm64`、`CGO_ENABLED=1`。
3. 使用 HarmonyOS clang 和 sysroot。
4. 以 `-buildmode=c-shared` 构建 `libohos.so`。
5. 输出到 `entry/libs/arm64-v8a/libohos.so`。
6. 将生成的 `libohos.h` 复制到 C++ include 目录。

修改 Go 代码后必须重新执行该脚本，再 Clean/Rebuild 后端 HAP。只重启应用不会更新已经打包的 `libohos.so`。

13.2 后端宿主 HAP

推荐使用 DevEco Studio：

```
Build -> Clean Project
Build -> Build Hap(s) / Run
```

如果本机已配置 `hvigor` CLI，也可以执行：

```
hvigor clean
hvigor assembleHap --mode module -p module=entry@default
```

后端当前目标 SDK 为 `6.0.1(21)`。

13.3 业务前端 HAP

```
Set-Location E:\new\my_harmony
hvigor clean
hvigor assembleHap --mode module -p module=entry@default
```

当前仓库的 `hvisorw` 是空占位文件，不能作为 wrapper 使用。如果本机没有全局 `hvisor` 命令，应通过 DevEco Studio Clean 后重新运行。前端当前目标 SDK 为 `6.0.0(20)`。

`.ets` 修改后必须重新构建 HAP，不能只杀进程再打开。

13.4 安装和运行顺序

1. 安装后端宿主 HAP。
2. 安装业务前端 HAP。
3. 启动后端宿主。
4. 等待数据库、18080、1127、1128 初始化完成。
5. 启动业务前端。
6. 验证 WebSocket 和 StGlobal 回读。

两个工程的签名配置包含本机路径和凭据，文档、日志和截图中不要复制这些值。团队交付时应改为安全的签名管理方式。

14. 快速联调教程

步骤 1：启动后端并检查 HTTP

启动 Go 后端宿主后，在设备本机访问：

```
http://127.0.0.1:18080/ping
```

预期包含：

```
{
  "framework": "gin",
  "message": "pong from Go"
}
```

PC 通过 HDC 查看时：

```
hdc fport tcp:18080 tcp:18080
```

然后访问：

```
http://127.0.0.1:18080/ping
http://127.0.0.1:18080/Api/Debug/GoLogs
```

步骤 2：启动前端并检查 WebSocket

查看前端日志：

[WS_DIAG] WebSocket 已连接

如果后端尚未启动，应看到连接错误和 3 秒后重连提示。

步骤 3：确认配置闭环

检查 GoLogs：

- 1. 收到 requestStGlobal。
- 2. 向目标 FSM 1279 发送 DISPLAY_ON。
- 3. 1128 收到 FSM_CONFIG。
- 4. 成功生成 StGlobal JSON。
- 5. 推送 topic=stglobal。

再检查前端：

- 当前子系统 configReady=true。
- 等级、出口、硬件开关显示设备回读值。

步骤 4：确认实时统计

检查：

- 1. 1128 持续收到 FSM_STATISTICS。
- 2. Go 每秒推送 statistics 和 homeStats。
- 3. 页面本批个数、重量、速度和效率变化。
- 4. 每 3 秒实时保存事务正常完成。

15. 分层排障

15.1 六层检查法

层	问题	证据
0. 进程	后端宿主是否活着	/ping
1. 前后端网络	HTTP/WS 是否连到同一地址	WEB_SERVICE_URL、WEBSOCKET_CLIENT_URL
2. Go 路由	Go 是否收到并识别请求	/Api/Debug/GoLogs
3. 设备下发	命令、destId、IP、端口、payload 是否正确	CTCP 发送日志
4. 设备回读	FSM 是否回推权威状态	1128 FSM_CONFIG/STATISTICS 日志
5. 前端状态	Mapper、门控、监听和页面是否消费	[WS_DIAG]、configReady、AppStorage

15.2 常见现象

现象	优先检查	常见原因
HTTP 和 WS 都失败	后端宿主、18080	后端未启动或端口占用
HTTP 正常，WS 不通	/ws/data 和 WS URL	地址被覆盖、连接未重建
WS 发送成功，Go 无日志	连接地址和 JSON Type	发到了错误进程或 Type 不匹配
commandAck.ok=true，设备没变	设备回读	TCP 发送成功不等于设备应用成功
页面先变新值，随后恢复旧值	StGlobal 回读	本地乐观值被设备旧快照覆盖

现象	优先检查	常见原因
statistics 一直收到，图表仍为 0	配置门控	StGlobal 未就绪或子系统 ID 不匹配
只有一个 FSM 不更新	fsmId/srcId	当前选择与回包来源不一致
速度突然归零	数据 stale	统计流中断，Go/前端按过期规则归零
历史查询偶发慢	ORM/WAL/事务	实时保存变慢或数据库锁等待
修改 Go 后行为没变化	动态库打包	未重编 libohos.so 或未 Clean HAP
修改 ArkTS 后行为没变化	DevEco 缓存	未重新构建和安装前端 HAP

15.3 关键日志

后端宿主 HiLog：

- HTTP/WebSocket server start result
- GORM init result
- TCP server start result
- CTCP servers listening
- WebSocket saveLevelData
- CTCP FSM_CONFIG
- CTCP StStatistics
- realtime save

业务前端：

- [WS_DIAG] WebSocket 错误
- [WS_DIAG] WebSocket 连接已关闭
- statistics 已到但配置未就绪
- 忽略不匹配 commandAck

16. 修改功能时的定位指南

需求	首先查看
修改页面布局	前端 pages/、 components/
修改主页等级表	pages/home/DataTablesTabBar.ets、 components/tables/
修改 HTTP 查询	前端 LocalWebApiClient.ets + 后端 database/*_api.go
新增 WebSocket Topic	后端发布常量/函数 + 前端 handleMessage()
新增设备控制命令	前端 ConfigSender/WS Client + 后端 WebSocket handler + Tcp/client
修改 CTCP 结构体	Go Tcp/protocol + 前端 Structures/Mapper + ABI 验证
修改等级保存	前端 Level/ConfigSender + 后端 websocket_grade_commands.go
修改硬件设置	前端工程设置页 + 后端 websocket_fsm_commands.go
修改实时统计	后端 ctcp_statistics.go/home_stats.go + 前端实时服务
修改实时落库	后端 realtime_save*.go + database/realtime_save*.go
修改数据库表	database/tb_*.go + migration + API + 前端 DTO

修改线协议结构体时必须同时验证：

1. Go unsafe.Sizeof。
2. 字段偏移和对齐。

3. C/C++ 下位机结构体版本。
4. JSON 字段名称。
5. ArkTS Mapper。
6. 固定长度名称的 GBK/UTF-8 转换。

17. 测试和验证命令

17.1 Go 单元测试

```
Set-Location E:\goTest\go\ohos
go test ./...
```

当前 Go 工程包含数据库 API、实时保存、WebSocket handler 和主页统计等测试。

17.2 后端专项工具

```
tools/db_ws_stability_test.py
tools/realtime_db_perf_test.py
tools/verify_weight_accumulate_save.py
tools/verify_project_config_backend.mjs
tools/verify_weight_state_logs.mjs
```

17.3 前端静态验证

前端 `tools/verify_*.mjs` 对主页表格、配置链路、主题、滚动、实时刷新等关键结构做源码级检查。修改相关模块后，应执行对应脚本并重新构建 HAP。

18. 当前代码中的注意点

1. 前端 `README.md` 和 `AGENTS.md` 仍包含旧的“前端直接启动 Native TCP/HTTP 29500”描述；当前 `EntryAbility.ets` 已经只启动 WebSocket 和 UIDataSync。
2. 当前前后端业务地址是 `18080`，不是旧文档中的 `29500`。
3. `Tcp/ARCHITECTURE.md` 同时描述“当前包结构”和“未来目标结构”，阅读时不要把目标目录当成已经完成。
4. 前后端是两个 Bundle，必须分别安装和启动。
5. 后端 Gin 只绑定 `127.0.0.1`，默认不向局域网暴露 HTTP/WebSocket。
6. WebSocket `CheckOrigin` 当前直接返回 `true`。只绑定 `localhost` 时风险有限，如果未来监听 `0.0.0.0`，必须重新设计来源校验和认证。
7. 签名配置、项目密码和设备密钥不能进入文档、日志或测试样例。

19. 一句话总结

这套系统的核心不是“ArkTS 调 Go 函数”，而是：

后端宿主通过 NAPI 拉起 Go runtime，业务前端通过 `localhost` HTTP/WebSocket 使用 Go 服务，Go 再通过 CTCP 与设备通信，并以设备回读作为最终权威状态。

排查任何问题时，都沿着下面这条链逐层取证：

```
页面 -> 前端状态 -> HTTP/WebSocket -> Go 路由和缓存
      -> CTCP 下发 -> FSM/IPM/WAM -> 1127/1128 回读
      -> Go JSON 推送 -> 前端 Mapper -> GlobalDataInterface -> 页面
```

20. 相关文档

- [鸿蒙 ArkTS 与 Go 后端联调实践](#)：培训和演示视角。
- [Go 后端 WebSocket 数据链路说明](#)：WebSocket、StGlobal 和 Statistics 的展开说明。
- [Go 在 HarmonyOS 上编译运行说明](#)：动态库、NAPI 和打包顺序。
- [StGlobal 字段映射说明](#)：Go、C++、ArkTS 字段和等级数据映射。
- [水果分选系统说明文档](#)：面向业务、实施和运维的系统概览。
- [Go 后端数据库与 WebSocket 稳定性测试](#)：稳定性验证方法和结果。
- [Tcp 后端模块化设计](#)：当前拆包状态和后续目标结构。